

# Framework Django

## Características generales

Es un framework basado en Python para la creación de aplicaciones Web. Incluye numerosas utilidades para realizar tareas habituales en una aplicación WEb, de modo que el programador no tenga que preocuparse de esos detalles

## Instalación

Lo primero será instalar Python, necesitaremos la versión 3.5 o superior. Python incluye la herramienta de gestión de paquetes pip, con la que instalaremos los diferentes componentes necesarios para trabajar con Django

## Entorno virtual

Para poder trabajar con Django, es conveniente disponer de un entorno virtual en el que se crearán los proyectos. Instalaremos el gestor de entornos virtuales a través del siguiente comando pip que ejecutaremos desde la línea de comandos, desde la carpeta de instalación de Python:

```
>pip install virtualenvwrapper-win
```

Seguidamente, crearemos nuestro primer entorno virtual al que llamaremos, por ejemplo, proyectodj1

```
>mkvirtualenv mientorno
```

## Instalación de django

Para instalar django ejecutaremos:

```
>pip install django
```

Después de instalarse, podemos verificar la versión que hemos instalado mediante el comando:

```
> django-admin --version
```

## Creación de un proyecto

Para crear un proyecto, una vez que ya tenemos django instalado, nos situamos en la carpeta donde queramos guardar dicho proyecto y tecleamos el siguiente comando:

```
> django-admin startproject proyectodj1
```

Donde "proyectodj1" es el nombre del proyecto. Tras ello, se creará una carpeta con este nombre, dentro de la cual se habrán creado una serie de elementos:

- `manage.py`. Utilidad de línea de comandos con la que podemos interactuar con el proyecto y realizar distintas tareas como crear una aplicación, iniciar el servidor, etc.
- `proyectodj1/`. Se trata de una subcarpeta con el mismo nombre del proyecto, que representa el paquete actual para el proyecto. Dentro de ésta, encontramos otros archivos `.py` como:
  - `__init__.py`. Fichero vacío, pero que indica a Python que este directorio es un paquete Python
  - `settings.py`. Datos de configuración para el proyecto django
  - `urls.py`. Direcciones para el proyecto django

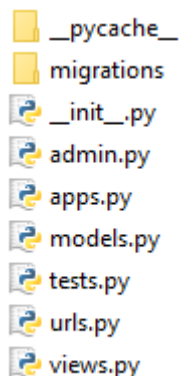
## Creación de una aplicación

Una aplicación Web forma parte de un proyecto. Un proyecto puede contener múltiples aplicaciones

Para crear una aplicación, por ejemplo "app1", nos situamos en la carpeta donde se encuentra `manage.py` y lanzamos el siguiente comando:

```
> python manage.py startapp app1
```

Tras el comando anterior, se creará una subcarpeta *app1* dentro de la carpeta principal del proyecto, cuya estructura será similar a la indicada en la siguiente imagen:



## Creación de una vista

Una vez creada la aplicación, lo siguiente será crear una vista dentro de ésta. Para ello, editamos el archivo `views.py` e incluimos el siguiente código:

```
from django.http import HttpResponse

def index(request):

    return HttpResponse("<h1>Bienvendio a mi primera pagina</h1>")
```

Las respuestas se generan con el objeto `HttpResponse` que tiene que ser importado desde la librería `django.http`. Para definir una respuesta asociada a la petición `index`, utilizamos el objeto `HttpResponse` para indicar la cadena HTML que queremos devolver con la petición.

A continuación, debemos mapear esta vista a una URL. en el archivo `urls.py` que está en la carpeta raíz de la aplicación debemos incluir lo siguiente:

```
from django.urls import path

from . import views

urlpatterns = [

    path("", views.index, name='index'),

]
```

Seguidamente, en el archivo `urls.py` que se encuentra en la raíz del proyecto, debemos importar `django.urls.include` y añadir una entrada que en la lista de `urlpatterns` que **asocie a la dirección /app1 las urls definidas en `app1.urls`**. El contenido de este archivo deberá quedar:

```
from django.contrib import admin

from django.urls import include, path

urlpatterns = [

    path('app1/', include('app1.urls')),

    path('admin/', admin.site.urls),

]
```

## Captura de datos de un formulario

Los datos de un formulario son capturados a través de una vista, que puede almacenarlos en el contexto de la aplicación para que una plantilla HTML los recupere y genere dinámicamente una vista.

En primer lugar, dentro de la carpeta raíz del proyecto crearemos una carpeta en la que guardaremos las plantillas HTML que forman las vistas de la aplicación. La primera será la página del formulario `inicio.html`:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Django Tutorial</title>
```

```
</head>

<body>

<h2>Formulario para Django</h2>

<form method="post" action="/mostrardatos/">

  <table>

    <tr>

      <td>Nombre:</td>

      <td><input type="text" name="nombre"/></td>

    </tr>

    <tr>

      <td>Email:</td>

      <td><input type="email" name="email"/></td>

    </tr>

    <tr>

      <td>Edad:</td>

      <td><input type="number" name="edad"/></td>

    </tr>

    <tr>

      <td>

        </td>

      <td>

        <button>Enviar datos</button>

      </td>

    </tr>

  </table>

</form>
```

```
</body>
```

```
</html>
```

Como vemos, la petición es enviada a la URL `"/mostrardatos"`. Por otro lado, crearemos una segunda página o plantilla que recogerá los parámetros y los mostrará en la página. A esta plantilla la llamamos `listado.html`:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Django Forms Tutorial</title>
```

```
</head>
```

```
<body>
```

```
<h2>Datos recibidos</h2>
```

```
<table>
```

```
    <tr>
```

```
        <td>Nombre:</td>
```

```
        <td><strong>{{ nombre }}</strong></td>
```

```
    </tr>
```

```
    <tr>
```

```
        <td>Email:</td>
```

```
        <td><strong>{{ email }}</strong></td>
```

```
    </tr>
```

```
    <tr>
```

```
        <td>Edad:</td>
```

```
        <td><strong>{{ edad }}</strong></td>
```

```
    </tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

Mediante las expresiones encerradas entre los símbolos `{{ y }}` volcamos los datos de contexto en la página.

Seguidamente vamos a crear una aplicación en la que definiremos la vista y las rutas, esta aplicación la llamaremos *appform*. Una vez creada, en el archivo `views.py` de la aplicación incluiremos el siguiente código:

```
#importing required packages

from django.http import HttpResponse

from django.template import loader

from django.views.decorators.csrf import csrf_exempt


#disabling csrf (cross site request forgery)

@csrf_exempt

def index(request):

    #if post request came

    if request.method == 'POST':

        #recuperamos los valores de la petición

        nombre = request.POST.get('nombre')

        email = request.POST.get('email')

        edad = request.POST.get('edad')


        #añadimos los valores a un json

        context = {

            'nombre': nombre,

            'email': email,

            'edad': edad

        }
```

```

#obtenemos la página en la que queremos mostrar los datos

template = loader.get_template('listado.html')


#renderizamos la página

return HttpResponse(template.render(context, request))

else:

    #si no es post, cargamos la página de inicio

    template = loader.get_template('inicio.html')

    return HttpResponse(template.render())

```

Por su parte, en el archivo urls.py de la aplicación asociaremos las urls a la vista:

```

from django.urls import path

from . import views

urlpatterns = [

    path('mostrardatos/', views.index),

    path('inicio/', views.index)

]

```

Además, en el archivo urls.py del proyecto habrá que incluir una referencia a este archivo para que tenga en cuenta las rutas de la aplicación:

```

from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('appform.urls'))

]

```

